

Multi-Agent Path Planning for Autonomous Vehicles in Parking Lots

Phi Nguyen and Zeyu Zhou

Abstract

We present a multi-agent path finding (MAPF) system for autonomous vehicles navigating shared parking lots from perimeter entry points to assigned parking spots without collision. The system implements two complementary algorithms: Conflict-Based Search (CBS), which finds provably optimal solutions, and Prioritized Planning (PP), which trades optimality for scalability. Both algorithms operate in a 4D state space $(x, y, \text{heading}, t)$ and model a kinematically realistic motion primitive set including 2-timestep turn maneuvers. We evaluate the system across four map scenarios of increasing difficulty with up to 24 agents. CBS is optimal for up to 8 agents (sub-10 ms) but becomes intractable beyond 12, while PP maintains millisecond-scale planning through all tested counts at a makespan cost of 4–46% above optimal.

1. Introduction

Autonomous vehicle (AV) parking is a coordination problem that grows rapidly in complexity with the number of vehicles. Vehicles must enter from limited access points, navigate shared lanes, avoid head-on conflicts at bottlenecks, and reach goal spots with a specific parking orientation — all without centralized real-time control.

We frame AV parking as a MAPF problem on a discrete grid where each agent occupies a single cell per timestep, carries an explicit heading, and must reach a designated goal cell facing the correct direction. We target two desirable properties that are often in tension: **solution optimality** (minimize total travel time) and **computational scalability** (plan for many vehicles in bounded time).

A key feature of our motion model is that turns are **not in-place**: a turn maneuver advances the vehicle into an adjacent cell before completing the heading change, consuming two timesteps and two grid cells. This better models the kinematic constraints of real cars compared to agents that can rotate freely on the spot.

Our contributions are: (1) a MAPF formulation with a car-like 2-timestep turn model and pose-based goal satisfaction; (2) implementations of CBS and PP in C++ with a BFS heuristic and 64-bit state encoding; (3) a suite of four map scenarios stress-testing coordination under progressively tighter bottlenecks; and (4) empirical evaluation revealing a sharp CBS tractability phase transition and PP’s scalability advantage.

2. Problem Formulation

2.1 State Space and Motion Model

The environment is a 2D grid $G = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$ where each cell is a **road**, **parking spot**, or **wall** (value $\geq \theta$). An agent's state is $s = (x, y, h, t)$ with heading $h \in \{E, N, W, S\}$ and timestep $t \in \mathbb{Z}_{\geq 0}$.

Each agent has five control actions. Let $(\Delta x, \Delta y)$ be the unit displacement in heading direction h :

Action	Steps	Effect
FORWARD	1	$(x, y, h, t) \rightarrow (x + \Delta x, y + \Delta y, h, t + 1)$
BACKWARD	1	$(x, y, h, t) \rightarrow (x - \Delta x, y - \Delta y, h, t + 1)$
WAIT	1	$(x, y, h, t) \rightarrow (x, y, h, t + 1)$
TURN_LEFT	2	$(x, y, h) \rightarrow (x + \Delta x^h, y + \Delta y^h, h) \rightarrow (x + \Delta x^h + \Delta x^{h_L}, y + \Delta y^h + \Delta y^{h_L}, h_L)$
TURN_RIGHT	2	$(x, y, h) \rightarrow (x + \Delta x^h, y + \Delta y^h, h) \rightarrow (x + \Delta x^h + \Delta x^{h_R}, y + \Delta y^h + \Delta y^{h_R}, h_R)$

All actions have uniform cost 1 per timestep consumed, so a turn costs 2. Turns are not in-place: the vehicle advances into the forward cell before the heading changes.

2.2 Goals, Conflicts, and Objectives

Agent i has start state (x_i^s, y_i^s, h_i^s) and goal (x_i^g, y_i^g, h_i^g) , where the required parking heading is inferred from the adjacent separator wall when omitted. The goal is satisfied only when the agent arrives at the correct pose with no future reservations at that cell by other agents, preventing false completion in transit.

Two paths conflict via a **vertex conflict** — agents i, j both at (x, y, t) — or an **edge conflict** — agents swapping cells between t and $t + 1$. We seek a joint plan $\Pi = \{\pi_1, \dots, \pi_N\}$ minimizing makespan $\max_i |\pi_i|$ subject to collision freedom, with CBS additionally minimizing sum-of-costs $\sum_i |\pi_i|$.

2.3 Assumptions

The environment is fully observable and static (no dynamic obstacles beyond agents). Motion is synchronous; actions execute instantaneously and deterministically. Each cell holds at most one agent and is large enough for one vehicle.

3. Approach

3.1 Shared Low-Level Planner: Space-Time A*

Both CBS and PP use **space-time A*** over (x, y, h, t) . For each agent we precompute a BFS distance map $d(x, y)$ from the goal, ignoring heading and time — this heuristic is **admissible** because no action reduces grid distance by more than 1 per timestep. States are packed into a 64-bit key for $O(1)$ closed-set lookup:

$$\text{key} = (t \ll 22) \mid (x \ll 12) \mid (y \ll 4) \mid (\text{phase} \ll 2) \mid h$$

where $\text{phase} \in \{0, 1\}$ distinguishes the mid-turn intermediate cell from a full state.

3.2 Conflict-Based Search (CBS)

CBS maintains a min-heap of **constraint-tree (CT) nodes**, each holding a constraint set C and one cost-optimal path per agent satisfying C . At each expansion, the joint solution is scanned for the first conflict. If none, the solution is optimal and returned. Otherwise, a conflict between agents i and j spawns two children — one adding a vertex or edge constraint on i , the other on j — and only the constrained agent replans via low-level A*. CBS is complete and optimal when the low-level planner is optimal. A hard cap of 10,000 CT nodes bounds runtime; the best solution found is returned if exceeded.

3.3 Prioritized Planning (PP)

PP plans agents sequentially in priority order σ (lower start y first, favoring interior agents). Each agent runs space-time A* against a growing **reservation set** $R \subset \mathbb{Z}^2 \times \mathbb{Z}_{\geq 0}$ of space-time points occupied by already-planned agents. Swap conflicts are blocked inline: a move from $(x_1, y_1) \rightarrow (x_2, y_2)$ at time t is rejected if $(x_2, y_2, t - 1) \in R$. Goal acceptance additionally requires no future entry $(x^g, y^g, t') \in R$ for $t' > t_{\text{cur}}$. If any agent fails Pass 1, their start cells are permanently added to R and all remaining agents are replanned (Pass 2 recovery), breaking deadlocks caused by stuck agents blocking interior paths.

4. Experimental Evaluation

4.1 Scenarios

We evaluate on four maps of increasing difficulty:

Map	Dimensions	Max Agents	Key Challenge
Main Parking Lot	28×64	144	Dense interior navigation
Gauntlet	16×24	24	Bidirectional 2-cell bottleneck
Two Lots	48×52	16 per side	Cross-lot travel via 3 roads
Two Lots Gauntlet	30×26	—	Compact 2-cell bottleneck variant

The **Gauntlet** is the primary benchmark: top agents are assigned bottom spots and bottom agents top spots, so all must cross a central 2-cell corridor in opposing directions — a maximally adversarial

coordination test. We sweep $N \in \{4, 8, 12, 16, 20, 24\}$ with both algorithms, validating all outputs with an independent conflict checker.

4.2 Results

Agents	CBS (ms)	PP (ms)	CBS makespan	PP makespan	Gap	CBS nodes
4	0.61	0.64	20	22	+10%	11
8	8.4	1.2	21	22	+4.8%	98
12	487	1.8	24	25	+4.2%	10,001
16	610	3.0	22	30	+36%	10,001
20	481	4.3	24	32	+33%	10,001
24	610	11	24	35	+46%	10,001

CBS is strictly optimal for $N \leq 8$ (sub-10 ms). At $N = 12$ it hits the node cap (487 ms) and its runtime plateaus near 610 ms for all larger counts — the cap cost dominates. PP scales near-linearly in N , staying under 12 ms through 24 agents and running exactly N low-level A* searches with no CT expansion. The root CT node already contains 2–52 conflicts at the Gauntlet (growing with N), confirming the head-on structure is adversarial to CBS’s branching. For $N \leq 8$, PP’s makespan overhead is small (5–10%); beyond the CBS tractability threshold the gap widens to 33–46%, as PP’s greedy priority ordering forces agents to wait serially rather than interleaving. At $N = 24$ PP’s two-pass recovery skips one agent, the first observed failure. Animated solutions on all four maps confirm valid, collision-free trajectories for both algorithms.

5. Limitations

CBS scalability. CBS is exponential in the number of conflicts and becomes unusable beyond ~8–10 agents in dense scenarios. The node cap bounds runtime but provides no guarantee on solution quality; the gap between capped and optimal solutions is unbounded in general.

PP incompleteness. PP is incomplete: a fixed priority ordering can deadlock even when a valid joint plan exists. The two-pass recovery mitigates but does not eliminate this — one agent is skipped at $N = 24$. Dynamic re-prioritization could recover more failures but is not implemented.

Turn geometry in narrow corridors. The 2-timestep turn occupies two grid cells simultaneously. In corridors exactly 2 cells wide, a turning vehicle blocks both cells mid-maneuver, potentially causing infeasibility that would not exist with in-place rotation.

Open-loop execution. Both algorithms produce offline plans with no replanning capability. Actuation noise, unexpected obstacles, or agent deviations invalidate the plan without any recovery mechanism. Real deployment would require an online reactive layer.

6. Conclusion

We presented a dual-algorithm MAPF system for autonomous vehicle parking with a kinematically realistic motion model. The 4D state space with explicit heading, BFS-precomputed admissible

heuristic, 2-timestep non-in-place turns, and two-pass PP recovery combine to handle real parking-lot geometry more faithfully than standard MAPF formulations.

Empirically, CBS and PP define a clear optimality-scalability tradeoff: CBS is optimal for small groups (up to 8 agents) while PP provides near-real-time planning for 20+ agents at moderate quality cost. The Gauntlet benchmark isolates the coordination challenge and reveals a phase transition at 12 agents where CBS’s exponential blowup becomes dominant.

Future directions include anytime CBS / ECBS for bounded-suboptimal solutions, dynamic priority reordering to eliminate PP deadlocks, continuous-time extensions for heterogeneous vehicle speeds, and online replanning for closed-loop execution.